

공세민

Se Min Kong

로봇과 소프트웨어를 연결하는 개발자

안녕하세요. 공세민입니다.

숭실대학교에서 소프트웨어를 전공했습니다.

현재 SSAFY Robotics Track에서 공부하고 있습니다.

센서 입력부터 제어 명령과 실제 동작까지 연결하며,
동작 조건과 실패 경계를 함께 살펴봅니다.

semin1224@gmail.com

github.com/SeMinKong



현재

SSAFY Robotics Track
2026.01 - 현재

학력

숭실대학교
소프트웨어학부
2020.03 - 2026.02

관심 분야

Computer Vision
Robotics / Physical AI

거주지

Suwon,
Republic of Korea

프로젝트

장비 연동과 실물 제어를 중심으로, 입력과 출력의 품질을 다룬 작업을 소개합니다.

쪽	프로젝트	역할	핵심 판단
3 - 7	THING	6인 팀 / 구동·기구 통합	기구 편차 / 초기 목표와 토크 순서
8 - 11	AQIS	2인 팀 / 팀장·서버·장비 통합	장비 연결 분리 / 좌표 갱신과 잡기
12 - 14	Briefit	6인 팀 / AI 담당	입력 정제 / 요약과 정보 보존
15 - 17	Brain MRI	개인 / 전처리·학습·통합 추론	라벨 변환 / 분류·분할 결합
18 - 20	Prompt Generator	개인 / 대화 서버·상태 관리	영역별 이력 / 수정 흐름
21 - 23	Alkkagi.io	개인 / 클라이언트·서버·물리	충돌·검침 보정 / 서버 입력 제한

THING · 텐던 로봇 핸드

기간

2026.07 - 08

팀·역할

6인 팀

모터 제어·기구 통합

사용 도구

DYNAMIXEL / U2D2

모터 점검·제어 스크립트

팀 기술

ROS 2 / MediaPipe

OpenCV / Jetson

Raspberry Pi 5



통합 조립

프로젝트 개요

카메라로 읽은 사람의 손동작을 텐던 로봇손으로 재현합니다. 손동작 인식부터 ROS 2 제어, 모터 구동과 관제까지 연결한 팀 프로젝트입니다.

담당 작업

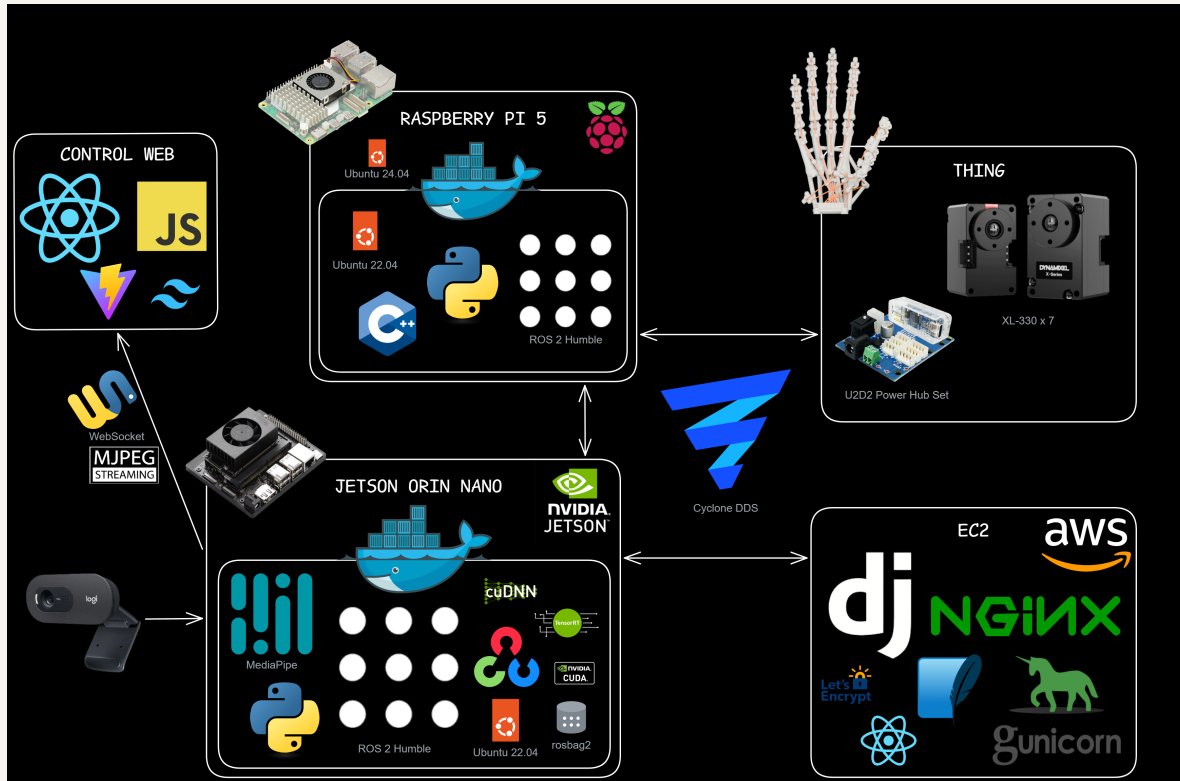
U2D2 통신 환경과 개별·원위치·정지 제어 스크립트를 작성하고, 아크릴 모터 고정부를 제작해 스프링·텐던과 조립했습니다. 소프트웨어의 위치 명령이 실제 손가락 동작으로 이어지는 구동부를 맡았습니다.

협업 범위

손동작 인식, ROS 2 명령 중재·guard, 관제와 데이터 기록은 팀의 협업 결과입니다. 본인 기여는 모터 제어와 기구 통합입니다.

THING · 손 자세를 모터 명령으로 변환

MediaPipe landmark → 7축 목표 → ROS 2 명령 선택·검사 → DYNAMIXEL 구동



구현 범위 인식·ROS 2 중재·guard·운영 드라이버는 팀 구현, 본인은 모터 점검과 기구 통합을 담당했습니다. DDS는 내부 통신이며, Jetson → EC2 기록 전송은 HTTPS입니다.

21개 landmark에서 7축 목표로

팀 인식 코드는 관절 굴곡 5축과 엄지 대립·외전 2축을 분리합니다. 굴곡은 관절각을 합성하고, 엄지는 손바닥 쪽으로 정규화한 거리와 평면 투영 각도로 계산합니다. deadband·저역통과 필터·프레임별 변화량 제한으로 목표의 흔들림을 줄입니다.

선택된 명령만 구동부로 전달

ROS 2 토픽으로 모방·원격 조작·수동 동작 명령을 분리하고 manager가 제어권을 선택합니다. guard는 상태·시각·축 범위·변화량을 검사합니다. 드라이버가 일반 손가락 축을 엔코더 값으로 바꾸고 엄지 기능 자세를 선택해 Sync Write로 전송합니다.

직접 구현한 모터 점검 경로

U2D2 통신과 개별·원위치·정지 스크립트를 작성했습니다. 이동 명령에서 현재 위치를 Goal Position에 기록한 뒤 Torque ON과 새 목표를 적용했습니다. 이 점검 도구로 구동부를 확인하고 아크릴 고정부·스플·텐던을 통합했습니다.

THING · 관절 기하에서 모터 제어 주기로

구현 원리 / 7축 자세 계산·ROS 운영 제어는 팀 구현 · 본인 기여는 모터 점검 도구와 기구 통합

01 관절 사이의 각도와 손바닥 기준으로 7축 목표 계산

21개 landmark에서 관절을 중심으로 두 벡터를 만들고 내적의 acos 로 각도를 구합니다. 굽힘량(180° -관절각)을 125° 로 나눠 0~1로 제한하고 PIP·DIP(엄지 MCP·IP)를 65%·35%로 합쳐 굴곡 5축을 만듭니다. 엄지 대립은 손끝-손바닥 중심 거리/손바닥 폭을 0.20~1.25로 역정규화합니다. 외전은 CMC→엄지 끝·검지 MCP 방향을 손바닥 평면에 투영한 각도 10~65°로 계산해 대립 성분의 혼입을 줄입니다. 입력은 영상 정규화 좌표이므로 정밀한 실제 관절각 측정값은 아닙니다.

굴곡·평면 투영 / clip: 0~1, n: 손바닥 법선

```
a=P_before-P_joint; b=P_after-P_joint
theta=acos(clamp(dot(a,b)/(|a|*|b|), -1,1))
bend=clip((pi-theta)/radians(125))
flex=clip(0.65*bend_near+0.35*bend_far)
v_plane=v-dot(v,n)/dot(n,n)*n
```

02 입력 흔들림을 제한하고 모터별 엔코더 목표로 변환

보정 구간을 0~1로 맞춘 첫 유효 표본을 필터 시작값으로 쓰고, 이후 deadband 0.02·저역통과 계수 0.25·프레임별 변화 상한 0.08을 적용합니다. 검지 입력 $q=0.5$ 이면 home 1740~closed 4300을 보간해 최종 목표 3020 pulse를 얻습니다. 엄지는 대립·외전의 네 기준 자세 중 가장 가까운 후보를 선택합니다. MIMIC은 후보가 명령 수신 3회 유지되고 거리 margin 0.1을 만족해야 바뀌 잦은 전환을 억제합니다. 자세 변경은 굴곡 여유 확보→필요한 중간 목표→대립·외전→굴곡 순서이며 위치 피드백으로 다음 단계를 결정합니다.

이후 표본의 필터·네 손가락 변환 · 의사코드

```
q=clip((raw-min)/(max-min))
if abs(q-prev)>0.02:
    q=prev+clamp(0.25*(q-prev), -0.08,0.08)
else: q=prev
goal=round(1740+q*(4300-1740)) # index
```

03 추론 도착 간격과 모터 쓰기 주기를 분리해 최신 목표 추종

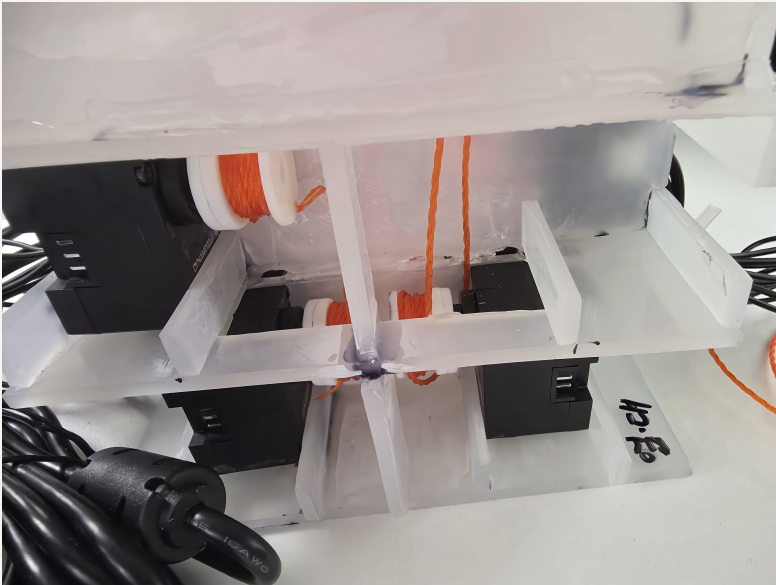
영상·landmark는 BEST_EFFORT·KEEP_LAST(1)로 과거 프레임 누적을 억제합니다. 쿨백의 최신 목표를 20Hz 타이머가 새 stamp·sequence로 발행하고, 250ms 이상 유효 표본이 없으면 캐시를 폐기합니다. manager·guard는 수신 즉시 제어권·순서·300ms 노후 명령·변화율을 검사합니다. 드라이버는 실제 경과 시간으로 이동량을 계산해 50Hz 쓰기에서 버스별 Sync Write, 20Hz 읽기를 수행합니다. 단일 spin의 직렬 I/O가 지연될 수 있어 설정 주기가 실측 보장은 아닙니다.

주기별 이동량 · dt는 초 / 실측 지터 결과 없음

```
dt=clamp(steady_now-last_update, 0,0.25)
pps=profile_velocity*0.229*4096/60
step=max(1,floor(pps*speed_limit*dt))
cmd+=clamp(goal-cmd, -step,step)
# latest target: publish 20Hz / write 50Hz
```

THING · 기구 편차와 제어 조건

같은 모터 위치 명령도 텐던 장력과 권취 방향에 따라 실제 손가락 자세가 달라집니다.



구동부 내부: 모터·스풀·텐던 연결



전원부 모터 고정부

위치 명령과 실제 굽힘

통신 응답이 정상이어도 원하는 자세가 나오지는 별도 확인이 필요합니다. 모터 점검과 실제 기구 동작을 나눠 확인했습니다.

재장착 조건의 보존

모터 고정부를 제작하고 스푼·텐던을 통합했습니다. 체결 방향과 텐던 경로를 사진·작업일지에 남겨 재장착 시 참고할 기준을 마련했습니다.

이동 명령의 토크 순서

이동 명령에서는 현재 위치 읽기 → 목표값 기록 → 토크 ON 순서를 적용합니다. 이전 목표값으로 갑자기 이동하는 것을 막기 위한 점검 코드입니다.

원위치와 정지

다회전 위치에서 가장 가까운 중앙각 좌표를 선택합니다. 키보드로 개별·전체 모터의 토크를 끄는 Torque OFF 경로도 마련했습니다.

축별 보정의 경계

중앙각 복귀만으로 손가락의 가동 범위가 보정되지는 않습니다. 당시 일지에서 축별 끝점 보정과 최대 가동 범위의 간섭 검증을 후속 과제로 남겼습니다.

THING · 실물 동작과 결과



캔 파지 시연

수상

SSAFY 공통 프로젝트 우수상

2026.08.10 / THING

[상장 보기](#)

팀 시연

손동작 모방과 손가락 순차 동작, 캔·부드러운 물체 파지를 시연했습니다. 인식과 제어 명령이 조립한 구동부를 거쳐 실제 손가락 동작으로 이어졌습니다.

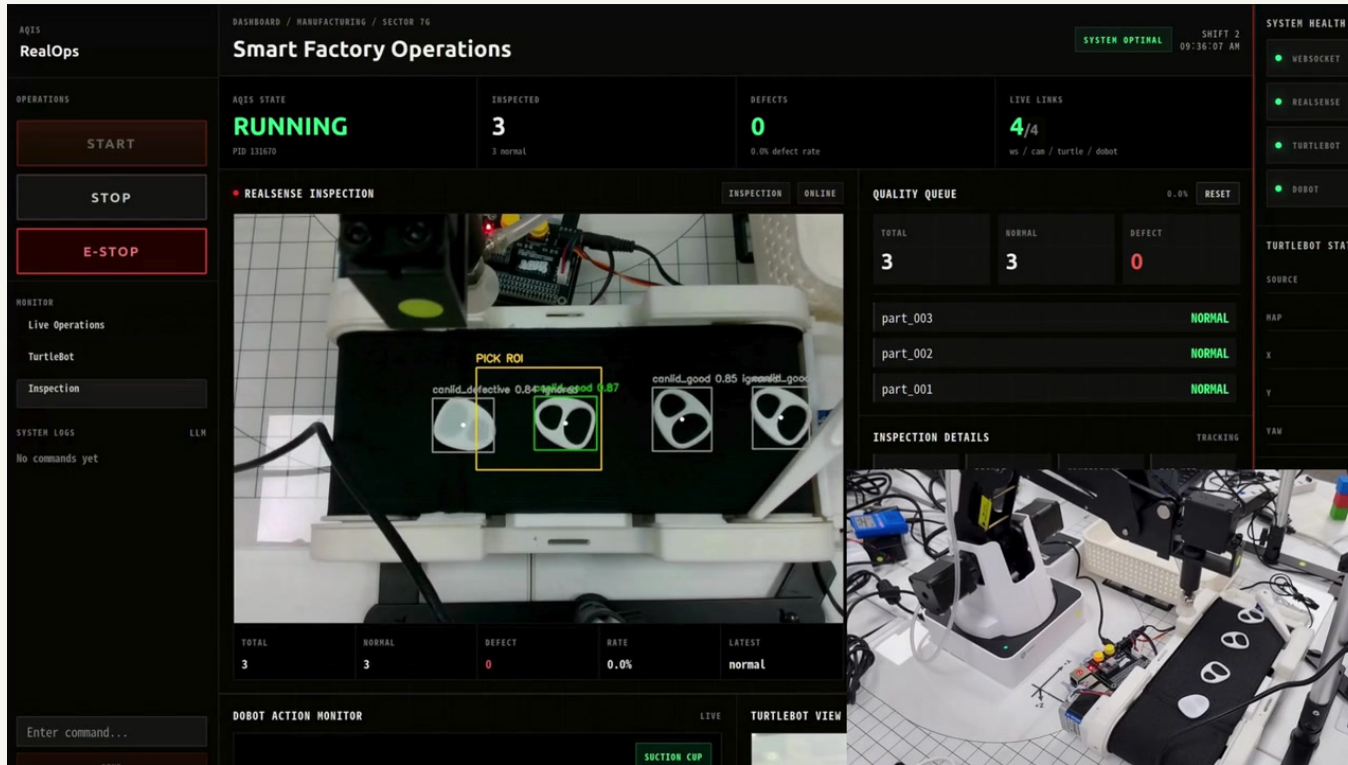
시연 이후의 판단

동작 시연은 시스템 연결의 근거입니다. 물체별 파지 성공률과 재조립 후 반복성을 판단하려면 장력·가동 범위와 시험 조건을 고정한 반복 측정이 필요합니다.

보정 전후 편차·반복 파지 성능의 정량 결과는 공개 기록에서 확인되지 않았습니다.

AQIS · 스마트 팩토리

검사, 컨베이어, Dobot과 RealOps 관제를 연결한 스마트 팩토리 시스템



RealOps · 실제 장비 시연

협업 팀원은 모델 학습·Roboflow·CAD·시뮬레이션을 담당했습니다.

2026.05 기획 / 06 본 개발

2인 팀 · 팀장

Full-stack & Robot Integration

프로젝트 개요

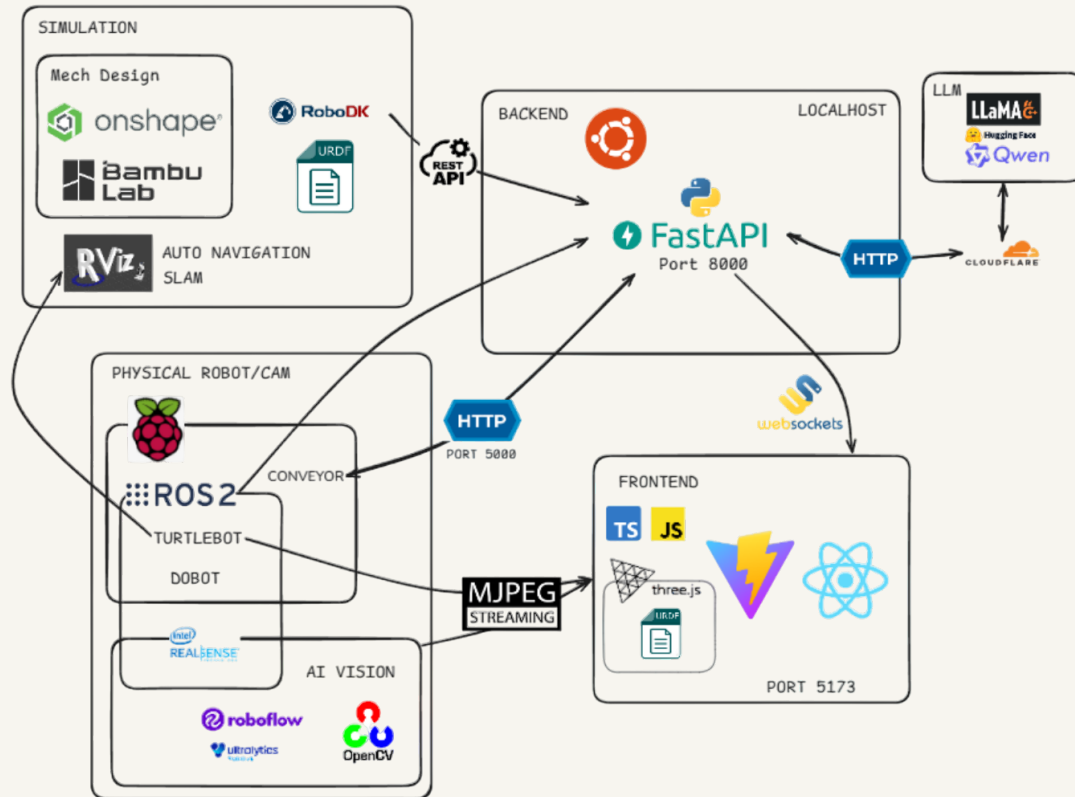
카메라의 검사 결과로 컨베이어를 제어하고 Dobot이 대상을 집어 분류합니다. RealOps에서 검사·작업 상태와 장비를 함께 관리합니다.

직접 맡은 일

React 관제와 FastAPI·WebSocket 서버, ROS 2 연결, 장비 adapter와 집기 시퀀스를 구현했습니다. 서버에서 검출 이벤트를 장비 명령으로 연결했습니다.

AQIS · 검출을 공정 제어로 연결

YOLO·깊이 카메라 → ROS 2 검출 이벤트 → FastAPI 상태 관리 → 장비 명령·WebSocket 관제



구현 범위 본인은 관제·서버·ROS 연결·장비 adapter·집기 시퀀스, 비전·모델 학습은 팀 구현입니다. 그림의 TurtleBot 자동 임무는 확장 설계이며 SLAM·카메라·상태 관제까지 구현했습니다.

영상의 검출 중심을 공간 좌표로

팀 비전 노드는 YOLO 검출 중심이 집기 ROI 안에 있는 후보를 남깁니다. 중심 주변 유효 깊이의 중앙값과 카메라 내부 파라미터로 픽셀을 3D 좌표로 역투영합니다. 라벨 누적과 동일 라벨·위치 이벤트의 재발행 제한을 거쳐 결과를 ROS 토픽에 보냅니다.

검출 형식을 통일해 상태를 갱신

직접 구현한 서버는 ROS 콜백의 JSON을 개별 검출로 펼치고 정상·불량 라벨을 공통 형식으로 정규화합니다. 중복을 거른 뒤 통계와 공정 상태를 갱신하고, asyncio 루프의 WebSocket으로 React 관제에 전달합니다.

장비 연결부와 집기 좌표의 분리

초기 계획의 실물 사용은 마지막 3일이었습니다. 공통 API 아래 Mock·실장비 adapter를 두어 관제 흐름을 먼저 연결했습니다. 정지 요청 뒤 갱신한 카메라 X/Y는 affine 계수·단위·오프셋을 적용해 Dobot 집기 목표로 변환합니다.

AQIS · 중복 집계와 집기 좌표 갱신의 분리

구현 원리 / 연속 검출을 수량으로 집계하는 조건과 정지 후 로봇에 전달할 좌표의 조건

01 움직이는 동일 검출을 중복 집계하지 않는 시간 창

연속 프레임의 검출을 모두 수량에 더하면 같은 물체가 반복 집계됩니다. 서버는 최근 8초 기록 중 label:result:is_defect가 같은 후보를 비교합니다. 명시적 ID 일치, IoU 0.5 이상, 중심 거리 70px 이하 중 하나라도 맞으면 중복으로 거절합니다. 이때 기록의 시각·위치를 갱신해 물체가 계속 관측되는 동안 최초 시각에서 8초가 지났다고 다시 집계하지 않게 합니다. 같은 불량에 양쪽 bbox.center가 모두 없어도 중복으로 봅니다. 객체 ID 추적기는 아닙니다.

ID·기하정보가 있는 경우 · 실제 설정 / 의사코드

```
key = (label, result, is_defect)
recent = records_within(now - 8.0)
duplicate = same_key and (same_explicit_id
    or IoU >= 0.5 or center_distance <= 70)
if duplicate: refresh(time, bbox, center)
```

02 중복 판정보다 먼저 정지 후 좌표를 선택하는 분기

RUNNING 중 첫 불량이 승인되면 컨베이어 정지를 요청하고 ready_at을 현재 시각+0.6초로 잡습니다. 이후 pending 분기를 중복 판정보다 먼저 실행해 집계에서 제외될 검출도 새 집기 좌표로 사용할 수 있게 합니다. 이벤트 timestamp가 준비 시점 이후이고 3초보다 오래되지 않은 불량 중 깊이가 있는 후보를 우선합니다. timestamp가 없으면 허용하고 최초 물체의 ID를 고정하지 않습니다. 정지 응답 실패를 자동 차단하지 않아 실제 정지 확인까지 보장하지는 않습니다.

pending 우선 경로 · stamp가 있는 경우 / 의사코드

```
if pending:
    xs = fresh_defects(ready_at, max_age=3)
    if now >= ready_at and xs:
        trigger(prefer_depth(xs)); clear_pending()
    return # before dedupe
```

03 카메라 좌표를 장비 좌표로 연결하고 실행 결과를 분리

팀 비전 노드는 검출 상자 중심을 깊이 격자로 환산하고 유효 깊이의 중앙값을 내부 파라미터로 역투영합니다. 본인 집기 서비스는 카메라 X·Y(m)에 보정 계수와 평행 이동을 적용하고 1000배 및 mm 오프셋으로 장비 목표를 만듭니다. Z는 고정 높이 또는 X·Y의 별도 affine 식을 사용하며 깊이가 누락 시 고정 좌표 경로가 있습니다. 통신은 start/stop/status adapter, 집기는 별도 프로세스로 분리합니다. 재개 설정이 켜져 있고 정상 종료 코드 0일 때 재개하며 센서로 파지 성공을 확인하는 구조는 아닙니다.

깊이 역투영 → m/mm 변환 · 식 축약

```
X=(u-cx)*d/fx; Y=(v-cy)*d/fy; Z=d
x_mm = 1000*(a11*X+a12*Y+tx) + ox
y_mm = 1000*(a21*X+a22*Y+ty) + oy
z_mm = 1000*affine_z(X,Y)+oz # enabled
if exit_code==0 and resume_enabled: resume()
```

AQIS · 좌표 갱신과 집기 제어

이동 중의 최초 좌표를 계속 쓰지 않고, 정지 요청 뒤 후속 검출로 집기 위치를 갱신합니다.

1

검출 수신

대상·작업 상태 확인

2

> 정지 요청·대기

설정 시간 대기

3

> 좌표 재취득

후속 검출·깊이 좌표 우선

4

> 집기·분류·재개

스크립트 정상 종료 시 재개

검출과 실행의 중복 분리

같은 대상이 여러 프레임에 나타나므로 객체 ID와 영역 겹침·중심 거리로 중복 집계를 억제합니다. 집기 작업 중에는 명령의 중복 실행도 막습니다.

상태와 시각에 따른 입력 선택

관제가 정지되면 검출에 따른 통계 갱신과 집기를 시작하지 않습니다. timestamp가 있는 검출은 준비 시점 이전이거나 허용 연령을 넘으면 제외합니다.

실물 완료의 확인 조건

설정 시간 대기와 스크립트 종료 코드는 실제 정지·파지 성공의 센서 확인이 아닙니다. 자동 흐름은 정지 명령의 실패 응답으로 다음 단계를 차단하지 않습니다.

누락 입력과 검증 범위

timestamp가 없으면 신선도 검사를 통과하고, 깊이가 없으면 고정 좌표를 사용합니다. 좌표 변환 기대값 불일치와 실물 반복 성능·통신 복구는 남은 검증 항목입니다.

중복·정지 상태·시각 조건: 원본 테스트와 후속 Mock 재현으로 확인. 실물 성공률·사이클 시간의 측정 결과는 없습니다.

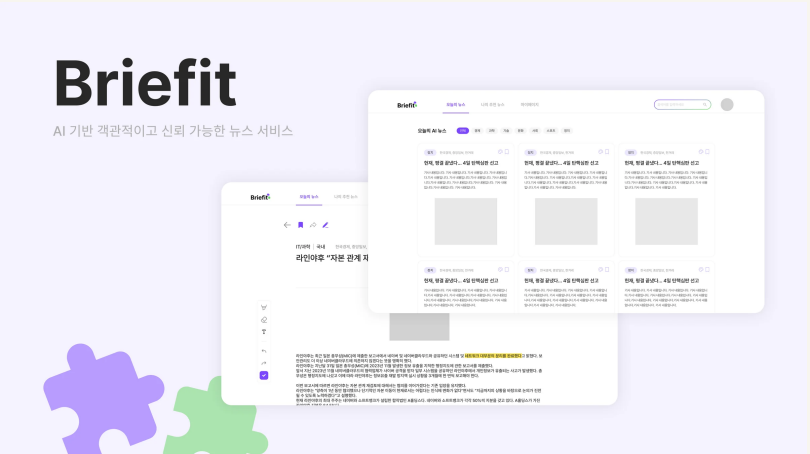
Briefit · 뉴스 요약과 출력 품질

여러 매체의 뉴스를 모아 요약을 제공하는 서비스 / 2025년 본인 AI 구현

기간
2025.05 - 09

팀·역할
6인 팀 / AI 2인
본인: AI 담당

사용 기술
Python / Crawl4AI
BeautifulSoup
Transformers / KoBART



서비스 UI · 팀 결과

직접 구현한 범위

기사 본문 정제와 배치 간 URL 중복 방지, KoBART 학습·생성·평가 스크립트를 구현했습니다. 긴 기사의 부분 요약·재요약과 생성문 후처리를 맡았습니다.

구현을 나눈 기준

수집 입력, 모델 생성, 출력 후처리를 분리해 기사 중복과 요약 누락·반복을 각각 점검할 수 있게 구성했습니다.

프로젝트 수상

2025 IT대학 소프트웨어 공모전 금상

2025.08.18 / Briefit

[상장 보기](#)

제15회 숭실 캡스톤디자인 경진대회 장려상

2025.10.01 / Briefit

[상장 보기](#)

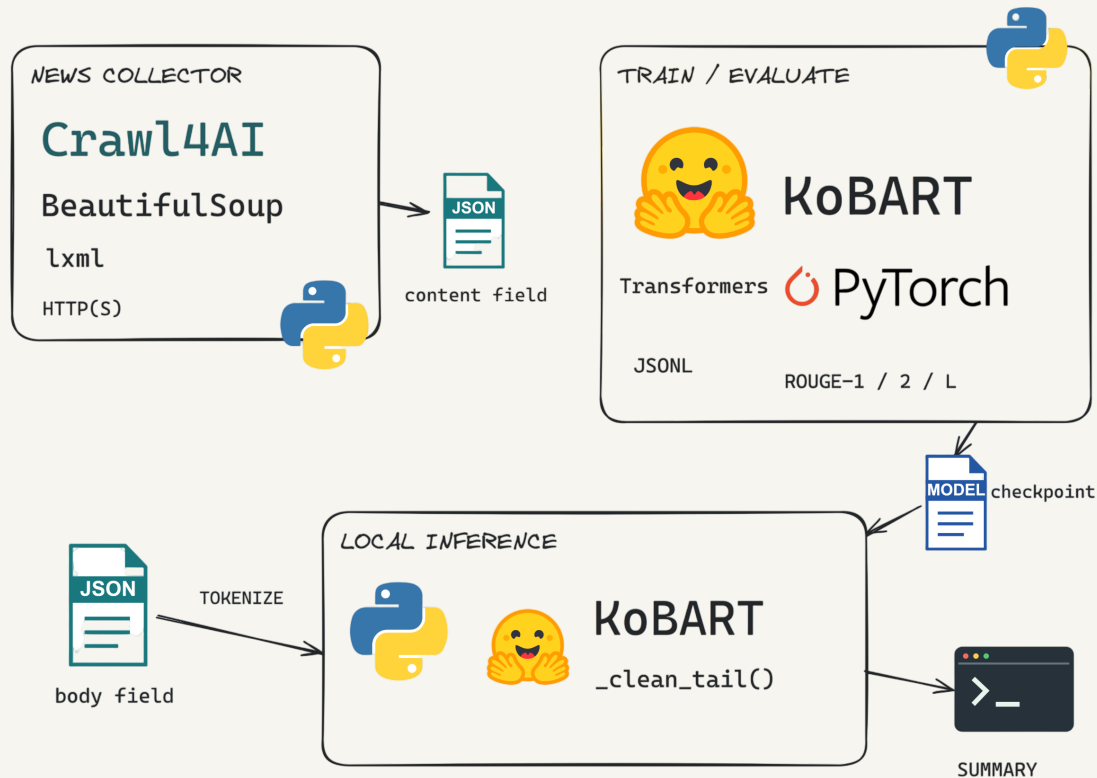
2025 IT 프로젝트 프로리그 장려상

2025.11.22 / Briefit

[상장 보기](#)

Briefit · KoBART 학습과 요약 생성

기사·기준 요약으로 학습 구성 → 기사에서 요약 토큰 생성 → 긴 입력 재요약·출력 정리



2025년 본인 구현 본문 정제·실행 내 URL 집합 공유부터 KoBART 학습·생성·평가 코드까지 담당했습니다. 학습 완료·ROUGE 점수 로그와 후처리 전후 품질 비교 결과는 확인되지 않았습니다.

기사와 기준 요약을 학습 쌍으로

사전학습 KoBART의 인코더는 기사 문맥을 표현하고 디코더는 요약 토큰을 생성합니다. 기사 text를 입력 토큰으로, 기준 summary를 정답 labels로 변환해 Seq2SeqTrainer에 전달합니다. 본인은 토큰화·학습 설정·평가 스크립트를 구성했습니다.

긴 입력은 부분 요약 뒤 재요약

생성 시에는 기사만 입력하고 beam search로 요약 후보를 탐색합니다. 긴 입력은 문단을 묶어 부분 요약한 뒤 결과를 이어 붙여 재요약합니다. 입력을 나누더라도 긴 문단과 재요약 입력은 토큰 제한으로 잘릴 수 있어 전체 문맥 보존을 보장하지 않습니다.

모델 생성과 규칙 후처리의 분리

별도 함수에서 생성문 끝의 반복·짧은 문장을 제거합니다. 정상 문장도 삭제될 수 있어 반복 감소와 정보 손실을 함께 확인해야 합니다. ROUGE는 기준 요약과 후처리 전 생성문을 비교하며, 최종 서비스 출력의 후처리 효과를 평가하지는 않습니다.

Briefit · 학습 텐서와 긴 기사 생성 경로

구현 원리 / 본인 2025년 KoBART 소스 기준 · 학습·서비스 생성·평가의 입력과 출력 조건

01 기사 입력과 기준 요약은 다른 길이의 학습 텐서로 구성

기사와 정답 요약은 역할과 길이가 달라 별도로 토큰화합니다. preprocess_fn은 기사 text를 384토큰, summary를 256토큰 한도로 자르고 각 max_length까지 패딩합니다. 기사 input_ids·attention_mask를 입력으로, 요약 토큰 ID를 labels로 Seq2SeqTrainer에 전달합니다. 사전학습 KoBART의 인코더가 기사 문맥을 표현하고 디코더가 기준 요약에 맞춰 생성하도록 학습 경로를 구성한 것입니다. 새 모델 구조나 손실함수를 직접 구현한 것은 아니며 학습 완료 로그·실측 점수는 확인되지 않았습니다.

학습 쌍 · 원본 동작을 줄인 의사코드

```
x = tokenize(text, max_length=384,
              truncation=True, padding=max)
y = tokenize(summary, max_length=256,
              truncation=True, padding=max)
Seq2SeqTrainer(input_ids=x, labels=y)
```

02 부분 요약을 중간 입력으로 만들어 긴 기사 재요약

긴 기사를 한 번에 자르면 뒤 문단이 생성 입력에서 빠집니다. smart_summarize는 1024토큰 이하이면 한 번 생성하고, 초과하면 문단을 누적해 부분 요약한 뒤 결과를 연결해 재요약합니다. 모든 생성은 beam 4·length_penalty 1.2를 사용합니다. 부분·최종 재요약의 출력 상한은 512토큰, 단일 생성 상한은 1024입니다. 다만 문단을 더한 뒤 길이를 검사하고 매 생성 입력도 1024토큰에서 잘리므로 문단 누적만으로 정보 손실을 모두 해결하지는 못합니다.

긴 입력 분기 · 괄호는 출력 max_length 설정

```
N <= 1024: generate(1024) -> clean_tail
N > 1024: paragraph_chunks(article)
           each -> generate(512) -> clean_tail
           join -> generate(512) -> clean_tail
generate: num_beams=4, length_penalty=1.2
```

03 문장 끝 정리 규칙과 ROUGE가 측정하는 대상을 구분

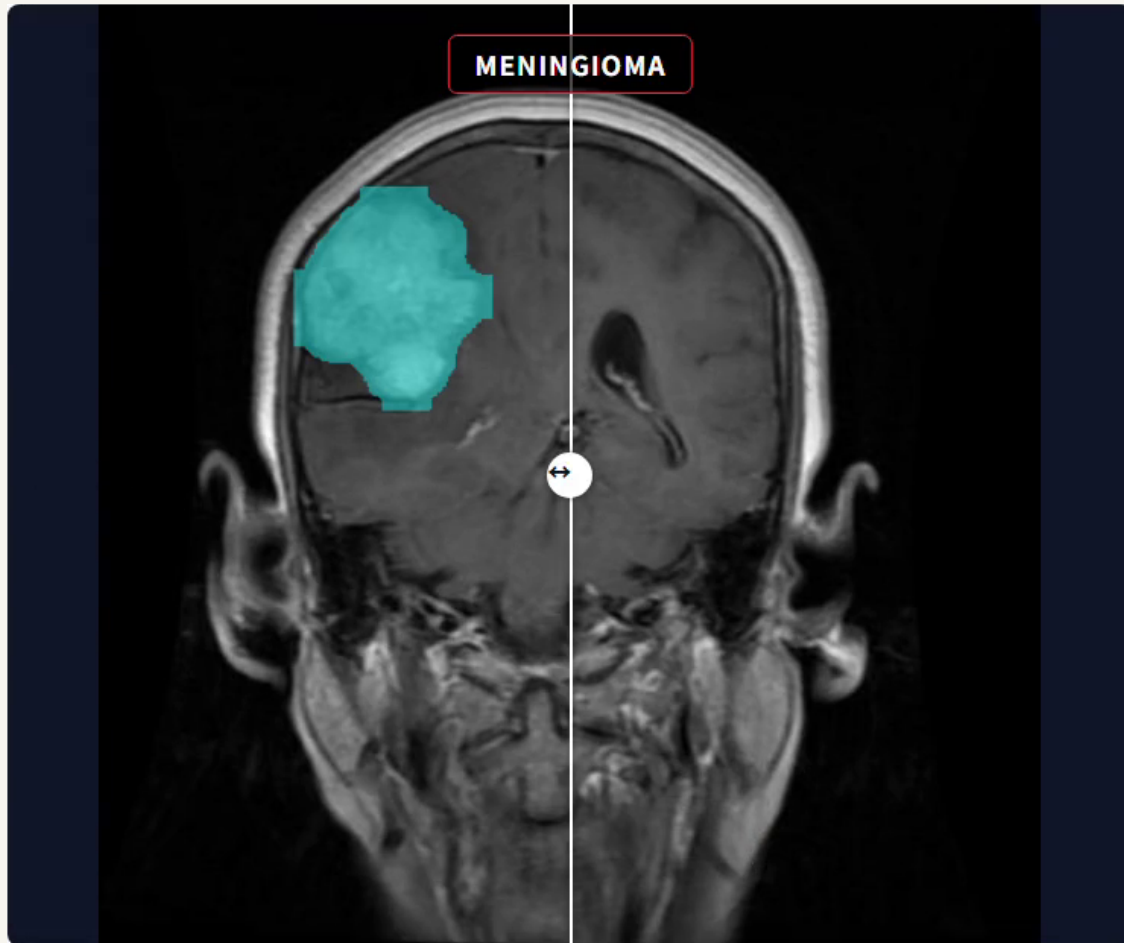
_clean_tail은 끝의 짧은 문장과 바로 앞 문장과 같은 반복을 while 조건으로 연속 제거합니다. 최종 출력뿐 아니라 각 부분 요약에도 적용돼 재요약에 들어가는 입력 자체가 바뀝니다. 규칙에 맞는 정상 문장도 지워질 수 있습니다. 별도 Evaluate.py는 기사 1024토큰에서 최대 128토큰을 beam 4로 직접 생성하고 디코딩 결과와 기준 summary의 ROUGE를 계산합니다. 이 경로는 긴 기사 재요약·후처리를 거치지 않으므로 서비스 출력의 정보 보존을 검증한 평가로 해석할 수 없습니다.

서비스 출력과 평가 대상 · 원본 경로 축약

```
while short_tail or repeated_previous:
    remove_last_sentence()
token_ids = tokenize(article).input_ids
eval = generate(token_ids[:1024], max_length=128)
score = ROUGE(decode(eval), reference)
```

Brain MRI · 종양 분류와 영역 분할

개인 프로젝트 / 2026.02 - 04 / Python · PyTorch · YOLO11 · OpenCV



분류·분할 통합 추론 · 원본 데모

프로젝트 개요

MRI의 종양 유형을 분류하고 영역을 분할하는 연구·학습용 프로젝트입니다. 분류 범주와 분할 위치를 같은 이미지에서 비교할 수 있도록 구성했습니다.

직접 구현한 범위

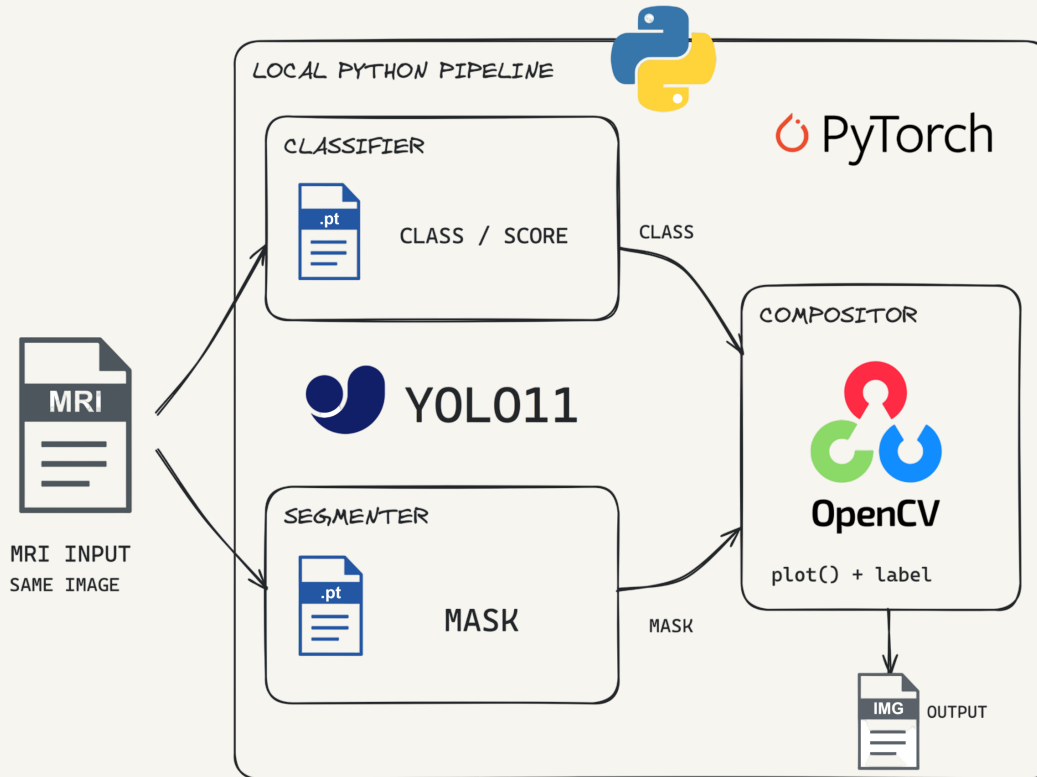
BRISC 이미지·마스크를 학습 형식으로 변환하고, 두 YOLO11 모델의 학습과 통합 추론 코드를 작성했습니다. 데이터 전처리부터 예측 시각화까지 연결했습니다.

학습 입력: 분류용 이미지 폴더 / 분할용 polygon label

출력: 분류 범주와 분할 영역의 통합 시각화

Brain MRI · YOLO11 분류·분할의 결합

이미지·마스크를 학습 형식으로 변환 → 두 모델에 독립 입력 → 범주와 분할 영역 시각화



평가 범위 학습에서 test를 val에 연결해 별도 독립 평가가 필요합니다. BRISC의 환자 단위 독립성은 확인할 수 없고 비종양 범주가 정상만 뜻하지는 않습니다. 임상 진단 검증은 완료하지 않았습니다.

유형과 위치를 서로 다른 모델로

YOLO11 분류 모델은 이미지 전체의 범주를, 분할 모델은 영역의 위치와 형태를 예측합니다. 클래스별 이미지 폴더와 이미지·polygon label 쌍으로 학습 입력을 나누고, 사전학습 cls·seg 모델을 각각 불러와 별도 가중치로 추론하도록 구성했습니다.

픽셀 마스크를 정규화 좌표로

마스크를 이진화하고 closing·opening으로 정리한 뒤 외부 윤곽을 추출합니다. 면적·점 수 조건을 통과한 꼭짓점의 x/y를 너비/높이로 나눠 polygon label로 저장합니다. 이 과정에서 작은 병변이나 내부 구멍이 사라질 수 있어 원본과 대조가 필요합니다.

같은 MRI의 두 출력을 함께 표시

같은 MRI를 두 모델에 각각 입력합니다. 분류의 최상위 범주·점수를 읽고 분할 마스크가 그려진 이미지 위에 표시합니다. 비종양 분류도 분할 추론을 차단하지 않으며, 두 결과가 다를 때 자동으로 판정을 보정하는 구조는 아닙니다.

Brain MRI · 픽셀 마스크를 분할 학습 라벨로

구현 원리 / 경계 정제·상대 면적 필터·좌표 정규화와 분류/분할 출력 결합

01 작은 틈과 잡음을 정제한 뒤 저장할 외부 경계 추출

픽셀 마스크를 YOLO가 읽는 polygon label로 바꾸려면 저장할 객체 경계를 정해야 합니다. mask_to_polygons는 회색조 값이 1보다 큰 픽셀만 전경으로 이진화하고 5×5 커널로 closing 다음 opening을 적용합니다. RETR_EXTERNAL로 외부 윤곽을 찾고 CHAIN_APPROX_SIMPLE로 직선 구간의 중간 점을 줄입니다. 작은 틈·돌출·잡음을 정리하는 방식이지만 내부 구멍은 표현되지 않습니다. 고정 5×5 커널은 이미지 해상도에 따라 상대적으로 다른 크기의 영역을 정리합니다.

마스크 → 윤곽 · 원본 계산 축약

```
B = 255 * (mask > 1)
K = ones(5, 5)
Q = opening(closing(B, K), K)
C = findContours(Q, RETR_EXTERNAL,
                CHAIN_APPROX_SIMPLE)
```

02 상대 면적과 점 수로 걸러 비율 좌표의 polygon label 저장

윤곽마다 점이 3개 이상이고 면적이 이미지 전체의 0.1% 이상인 경우만 남깁니다. 점의 x는 너비 W, y는 높이 H로 나눠 class_id 뒤에 소수점 여섯 자리 좌표쌍을 기록합니다. 클래스는 마스크 파일명의 중앙 코드에서 읽고 같은 basename의 txt로 저장합니다. 크기가 다른 영상의 위치를 비율로 통일하면서 작은 조각·다각형을 만들 수 없는 윤곽을 거르는 방식입니다. 면적은 윤곽 면적이며 작은 실제 병변도 제거될 수 있어 원본 마스크와 대조가 필요합니다.

라벨 채택 조건·표현 변환 · 의사코드

```
keep = len(contour) >= 3 and
        contourArea(contour) >= 0.001*W*H
x_norm = x / W; y_norm = y / H
label = class_id x1_norm y1_norm ...
write(basename + ".txt", precision=6)
```

03 유형과 영역을 독립 추론하고 한 이미지에 결합

분류에는 클래스별 이미지 폴더, 분할에는 이미지·polygon label 쌍을 사용해 yolo11m-cls와 yolo11m-seg를 따로 학습하도록 구성했습니다. 추론 함수는 같은 원본을 분류 다음 분할 모델에 각각 입력합니다. 분류 top1 범주·확률을 구하고 분할 plot 위에 문구를 그려 유형 판단과 영역 예측을 함께 비교할 수 있게 합니다. 비종양 분류도 분할을 실행하며 불일치를 자동 보정하지 않습니다. test를 val에 연결한 구성으로 독립 평가나 임상 검증이 완료됐다고 주장할 수 없습니다.

통합 추론 · 순차 호출 / 의사코드

```
p = cls_model.predict(image).probs
category, score = top1(p)
seg = seg_model.predict(image)
output = putText(seg.plot(),
                 category, score)
```


Prompt Generator · 프로젝트 설계 도우미

아이디어를 영역별 질문으로 구체화하고 결과를 하나의 문서로 모으는 도구

기간

2026

팀·역할

개인 프로젝트

대화 서버·상태 관리

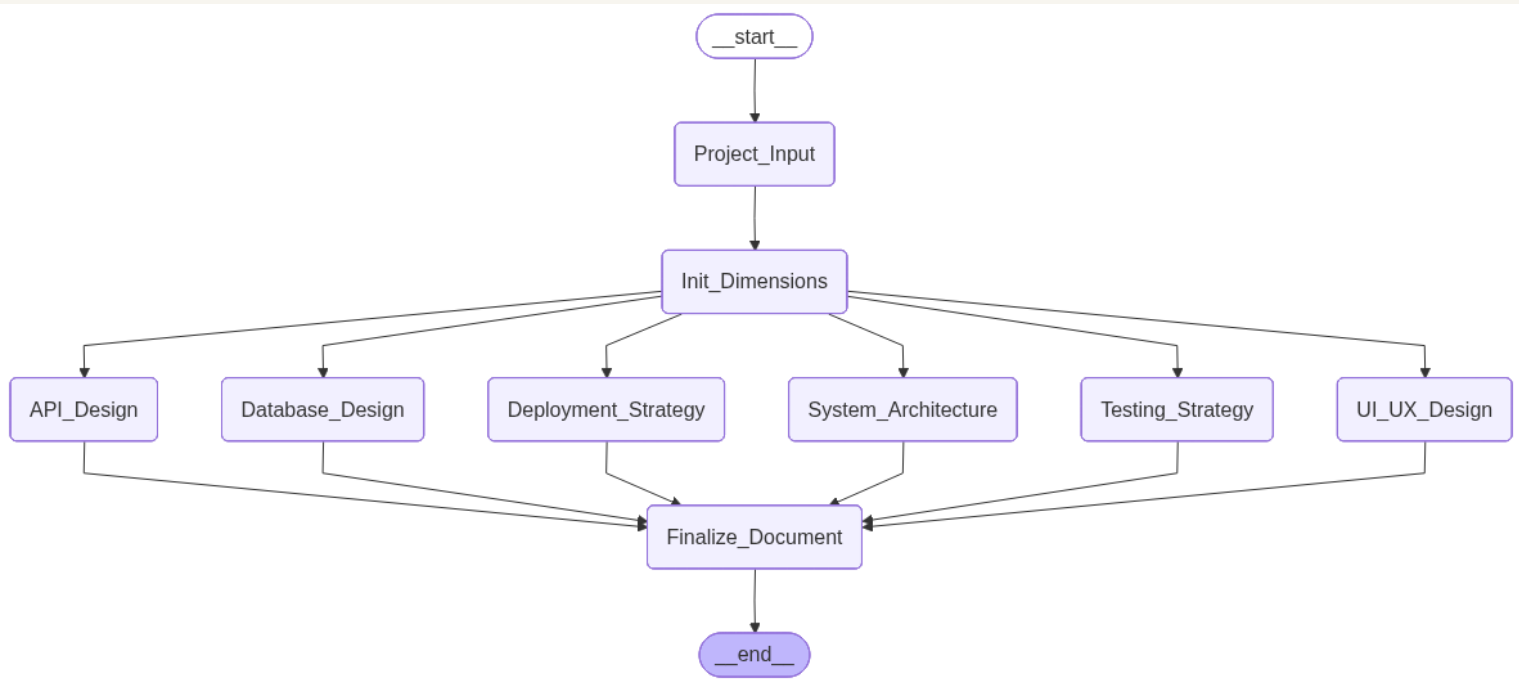
웹 UI

사용 기술

Python / FastAPI

WebSocket / LangChain

Solar Pro



초기 설계 흐름 · 원본

프로젝트 개요

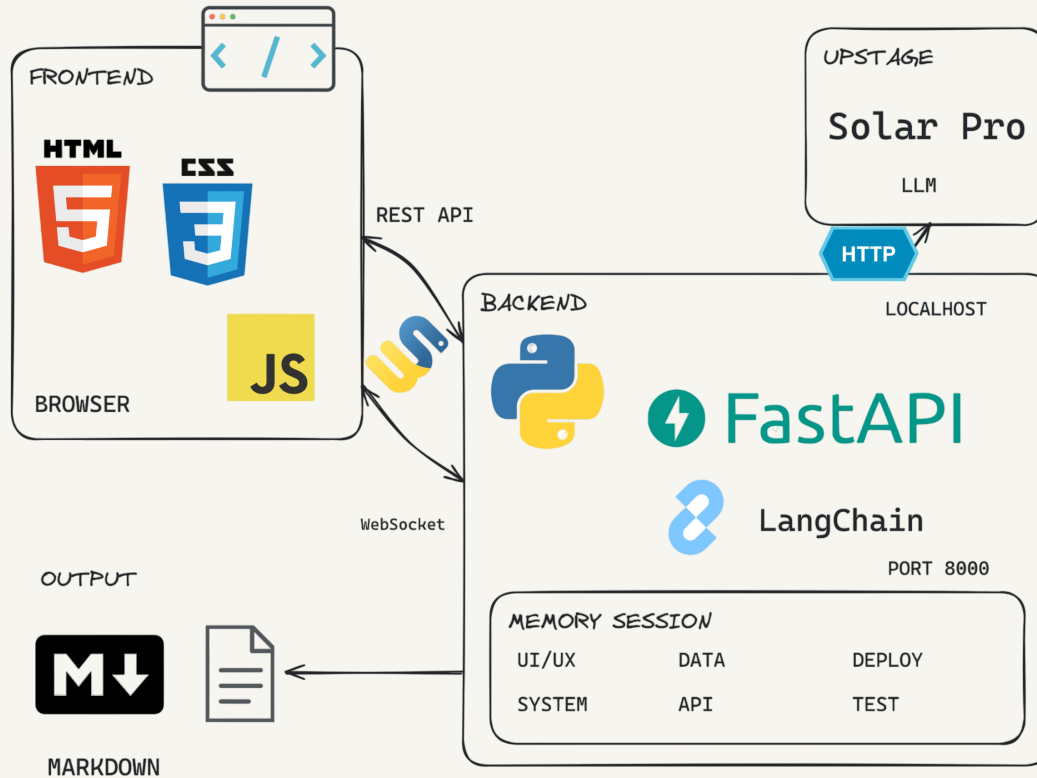
프로젝트 아이디어를 입력하면 화면·API·데이터 등 영역별 질문으로 요구사항을 구체화합니다. 각 영역의 결과를 하나의 설계 문서로 모읍니다.

직접 맡은 일

대화 서버와 웹 UI를 연결하고, 질문·답변에서 결과 문서 생성까지 진행 상태를 관리했습니다. 완료된 영역을 추가 대화로 수정하는 흐름도 구현했습니다.

Prompt Generator · 문맥 구성과 대화 상태

프로젝트 입력 → 영역별 지침·이력 → Solar Pro 응답 → 상태 갱신 → 설계 문서 합성



영역별 이력으로 다음 질문 구성

UI·API·DB·구조·테스트·배포의 여섯 영역을 나눕니다. 호출마다 영역 지침·프로젝트 설명·해당 대화 이력을 LangChain 메시지로 묶어 Solar Pro에 전달합니다. 첫 질문은 `asyncio.gather`로 병렬 실행하고 FastAPI가 WebSocket으로 응답을 전송합니다.

응답과 서버 상태를 함께 관리

영역별 라운드·이력·결과를 저장합니다. 처리 라운드가 3 이상이고 응답에 `[GENERATE_PROMPT]` 태그가 있으면 완료로 전환하며 추가 대화로 수정할 수 있습니다. 입력·응답은 호출 성공 뒤 기록하고, 처리 대상 오류는 라운드를 되돌리고 대기 상태로 전환합니다.

영역별 결과에서 최종 문서로

생성 결과가 있는 영역의 프롬프트와 프로젝트 설명을 모아 별도의 LLM 호출로 문서를 합성합니다. 서버는 하나 이상의 영역에 결과가 있으면 해당 부분만으로도 합성을 허용합니다. 웹 UI부터 대화 상태 전환·최종 문서 생성까지 직접 연결했습니다.

검증 범위 완료 조건은 생성 품질의 판정이 아닙니다. 처리 오류의 자동 재시도는 없고, 메모리 세션은 연결 종료 시 삭제됩니다. 재접속 복구와 요구사항 반영률·생성 품질의 평가 결과는 없습니다.

Prompt Generator · 문맥 구성과 완료 판정

구현 원리 / 영역별 질문 문맥을 서버에서 구성하고 호출 성공·완료·최종 합성을 따로 처리

01 영역별 문맥을 매 호출에서 재구성해 대화 혼입 방지

UI/UX·아키텍처·DB·API·배포·테스트 여섯 영역은 history·round·status를 각각 보관합니다. 매 호출에서 영역·라운드 지칭, 프로젝트 설명, 해당 영역의 이전 대화, 새 입력 순으로 LangChain 메시지를 구성합니다. 공통 프로젝트 맥락은 전달하면서 다른 영역의 대화는 섞지 않는 방식입니다. 첫 질문은 asyncio.gather로 병렬 시작하고 동기식 ChatUpstage(solar-pro).invoke는 to_thread로 실행해 LLM 응답을 기다리는 동안 이벤트 루프를 막지 않도록 했습니다.

입력 메시지·호출 경계 / 새 사용자 입력은 있을 때만

```
messages = [domain_system(round),
             project_description, *domain.history,
             *optional_user_message]
reply = await to_thread(llm.invoke, messages)
initial = await gather(*domain_questions)
```

02 모델 응답의 형식과 서버 상태 변경 조건을 분리

첫 질문도 라운드 1로 세고 호출 전에 round를 올립니다. 3라운드 이상이며 응답에 [GENERATE_PROMPT]가 있어야 완료로 인정하고 첫 태그 뒤 문자열을 generated_prompt로 저장합니다. 입력·응답은 호출 성공 뒤에만 이력에 추가합니다. RuntimeError·ValueError·OSError가 나면 round를 되돌리고 pending으로 복귀해 실패한 입력을 성공 이력에 섞지 않습니다. 미완료 성공은 in_progress를 유지하고 자동 재시도는 없습니다. 완료 태그는 문서의 품질 점수가 아닙니다.

성공·완료·처리 오류 · 의사코드

```
round += 1; status = "in_progress"
on_success: append_turn(user_if_present, reply)
done = round >= 3 and
        "[GENERATE_PROMPT]" in reply
on_handled_error: round -= 1; pending()
```

03 저장된 영역 결과를 수집해 별도 모델 호출로 최종 합성

최종 문서는 질문 이력을 단순 연결하지 않습니다. 비어 있지 않은 generated_prompt를 영역명 아래 모아 프로젝트 설명과 함께 별도 LLM 호출에 전달하고, 영역별 결과와 연결 관계를 담은 한국어 Markdown을 요청합니다. 저장 결과가 1개 이상이면 합성할 수 있고 완료 후 수정 대화도 허용합니다. 수정 중 실패해도 이전 generated_prompt는 남으므로 최종 수집 기준은 현재 status가 아니라 결과 존재 여부입니다. 세션은 연결 종료 시 삭제되며 문서 정합성을 자동 평가하는 단계는 없습니다.

결과 수집·합성 · 의사코드

```
sections = [(d.name, d.generated_prompt)
             for d in dimensions if d.generated_prompt]
if sections:
    final = synthesize(project, sections)
# existing result can survive a failed edit
```

Alkkagi.io · 실시간 알까기

여러 플레이어가 같은 보드에서 돌을 밀어내며 대전하는 웹 게임

기간

2026.03 - 04

팀·역할

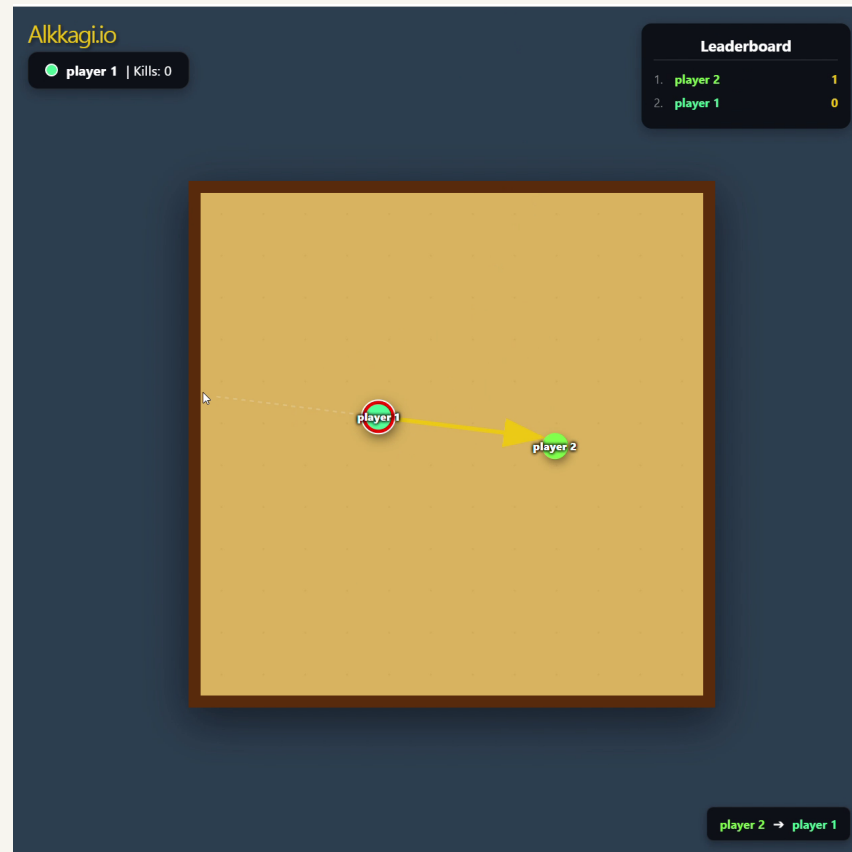
개인 프로젝트

웹 UI·서버·물리 계산

사용 기술

React / TypeScript

Node.js / Socket.IO



실제 플레이

프로젝트 개요

같은 보드에 접속한 플레이어가 돌을 밀어내며 대전하는 웹 게임입니다. 돌의 움직임과 충돌 결과를 실시간으로 공유합니다.

직접 맡은 일

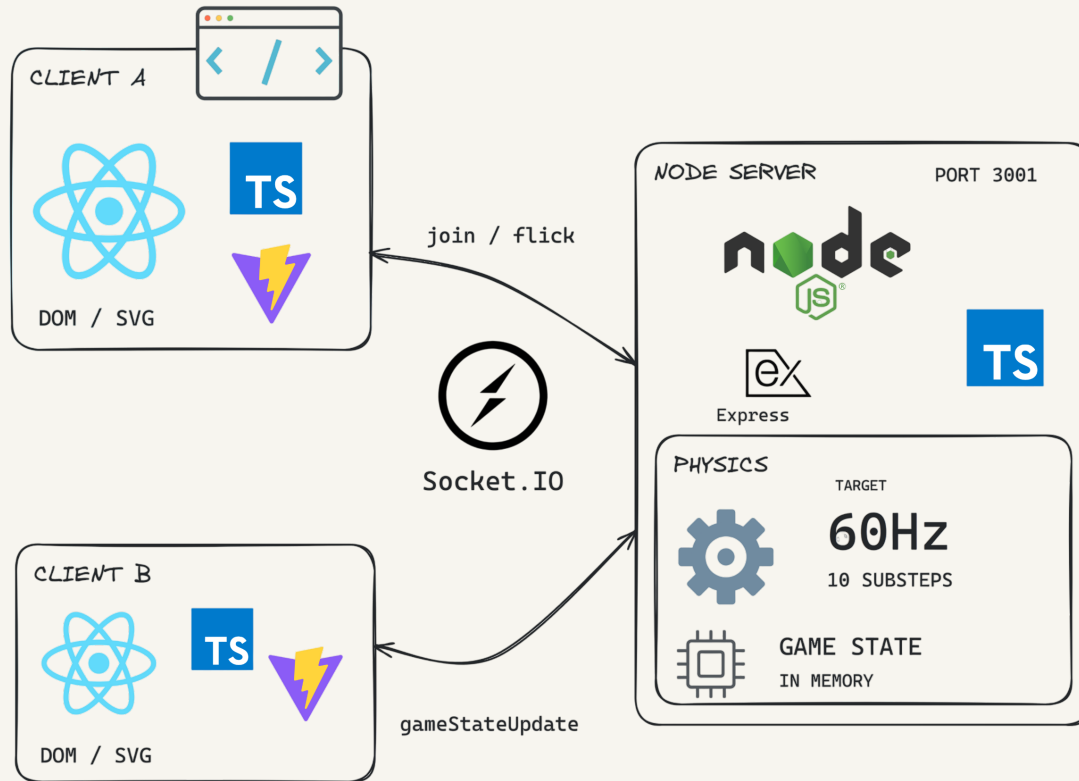
React 조준 UI와 Socket.IO 통신, 서버의 게임 상태·물리 계산을 구현했습니다. 충돌·마찰·검침 보정은 TypeScript로 작성했습니다.

게임 규칙의 실행 위치

클라이언트는 발사 입력을 보내고 서버가 기준 상태를 계산합니다. 입력 제한과 물리 규칙을 모든 플레이어에게 같은 기준으로 적용합니다.

Alkkagi.io · 서버 물리와 상태 동기화

드래그 벡터 → Socket.IO 발사 입력 → 이동·마찰·충돌 계산 → 모든 클라이언트에 상태 전송



개인 구현·검증 범위 웹 UI·통신·물리 함수를 직접 작성했습니다. 60Hz는 갱신 설정이며 부하·지속 프레임률의 실측값은 없습니다. 연결 종료 시 플레이어를 정리하며 재접속·서버 재시작 복구는 없습니다.

서버에서 입력 검증과 상태 계산

React의 드래그를 속도 벡터로 바꿔 flick 이벤트로 보냅니다. 서버는 발사 간격·최대 속도를 제한하고 질량을 반영해 속도를 정합니다. 위치·속도·반지름·질량은 서버 메모리에 두며, 갱신 후 gameStateUpdate를 배포해 클라이언트가 화면을 그립니다.

겹친 위치와 충돌 속도를 분리

중심 거리와 반지름 합으로 겹침을 찾고, 겹친 거리는 두 돌에 절반씩 나눠 위치를 보정합니다. 충돌 방향으로 이미 멀어지면 충격량을 더하지 않습니다. 접근하는 돌은 상대 속도·반발계수·역질량으로 충격량을 계산해 각각의 속도를 갱신합니다.

소단계에서도 감속 비율 유지

한 번의 갱신을 10개 소단계로 나눠 이동·충돌을 계산합니다. 마찰 계수에 소단계 비율을 지수로 적용해 분할 횟수만큼 감속이 중복되지 않게 했습니다. 보드 이탈 시 충돌 기록으로 점수·재배치를 처리하고 돌의 반지름·질량을 갱신합니다.

Alkkagi.io · 입력 제한과 충돌 계산의 실행 순서

구현 원리 / 서버가 입력을 속도로 변환하고 10개 소단계의 위치·마찰·충돌 계산 후 상태 전송

01 입력 벡터를 제한하고 질량으로 나눠 서버 속도 생성

클라이언트 flick를 곧바로 돌의 속도로 사용하지 않습니다. 접속자의 돌을 찾고 500ms 이내 재입력을 거절한 뒤 벡터 크기를 보드 폭의 45%로 제한하고 질량으로 나눕니다. 질량은 $1 + 0.05 \times \text{킬}$ 수입니다. 보드 600·입력 (300,400)·질량 1.5를 대입하면 크기 500을 270으로 줄인 (162,216)에서 최종 속도 (108,144)를 얻습니다. 이는 소스 식의 계산 예시이며 실험값은 아닙니다. 속도는 실제 초 단위가 아니라 서버 갱신 단계에서 사용하는 값입니다.

입력 크기 상한·질량 반영 · 원본 식 축약

```
cap = 0.45 * boardSize; q = length(input)
limited = input * min(1, cap/q) # q > 0
mass = 1 + 0.05 * kills
v = limited / mass
(300,400) -> (162,216) -> (108,144)
```

02 10개 소단계에서 전체 감쇠량을 유지하며 충돌 검사

서버 갱신 한 번을 10회로 나누고 매 단계에서 $p += 0.1v$, $v *= 0.8$ 의 0.1제곱을 적용한 다음 충돌을 검사합니다. 각 소단계에 0.8을 그대로 곱하면 전체 감쇠가 0.8의 10제곱이 되므로 지수에 단계 비율을 넣습니다. 충돌·정지 절삭이 없으면 10회 후 속도는 시작의 0.8배입니다. 각 축 속도의 절댓값이 0.1 미만이면 0으로 만들어 잔류 이동을 끊습니다. 10회가 끝난 뒤 전체 상태를 gameStateUpdate로 전송합니다. 1000/60ms는 타이머 설정이며 실측 유지 프레임률은 아닙니다.

한 서버 갱신의 순서 · 의사코드 / dt 보정 없음

```
repeat 10:
    p += 0.1*v; v *= 0.8**0.1
    zero_small_components(threshold=0.1)
    collide(); score_and_respawn_outside()
broadcast("gameStateUpdate", gameState)
```

03 겹침은 위치로, 접근 중인 충돌 반응은 충격량으로 보정

중심 거리 D가 두 반지름 합보다 작으면 겹침의 절반을 양쪽 위치에 반영합니다. 법선 n은 두 중심을 잇는 단위 벡터이고 $vn = (v_2 - v_1) \cdot n$ 은 법선 상태 속도입니다. vn이 양수이면 이미 멀어지는 중이므로 위치만 보정하고 충격량은 생략합니다. 접근 중에는 반발계수 0.7과 역질량 합으로 j를 구해 각 속도에 질량별로 반영합니다. 같은 겹침 보정량을 적용해도 속도 변화는 질량에 따라 달라집니다. 오른쪽 식은 $0 < D < r_1 + r_2$ 인 일반 충돌의 계산입니다.

일반 충돌의 위치·속도 계산 · 원본 식 축약

```
n = (p2 - p1) / D; overlap = r1 + r2 - D
p1 -= n * overlap / 2; p2 += n * overlap / 2
vn = dot(v2 - v1, n); if vn > 0: continue
j = -(1 + 0.7) * vn / (1/m1 + 1/m2)
v1 -= (j/m1) * n; v2 += (j/m2) * n
```

연락처

Se Min Kong

공세민 · 소프트웨어 개발자

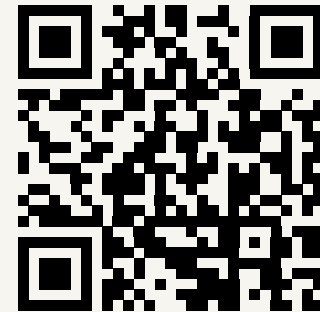
이메일

semin1224@gmail.com

GitHub

github.com/SeMinKong

웹 포트폴리오



seminkong.github.io/SeMinKong_Web/

프로젝트 영상과 상세 설명

이력서와 상장 전시